DSC 190/291 Assignment 6

Detailed Write-Up

Dhruv Patel

May 19, 2026

Contents

1. Part A: Boosting Sparse Linear Predictors and the ℓ_1 Margin

This problem continues the sparse-linear-model theme from Homeworks 4 and 5. Let $\varphi : \mathcal{X} \to \{-1, +1\}^d$ be a fixed feature map. For each $j \in [d]$ and $\sigma \in \{-1, +1\}$, define

$$b_{j,\sigma}(x) = \sigma\varphi_j(x),$$

and let

$$\mathcal{B} = \{b_{j,\sigma} : j \in [d],\ \sigma \in \{-1, +1\}\}.$$

For a $\{-1, +1\}$-valued predictor b, its edge under a distribution Q is

$$\frac{1}{2} - L_Q(b) = \frac{\mathbb{E}_Q[yb(x)]}{2}.$$

Assume $\mathcal{D}$ is realizable with margin by an s-sparse predictor: there is $w^\star \in \mathbb{R}^d$ with $\|w^\star\|_0 \le s$ and

$$y\langle w^\star, \varphi(x)\rangle \ge 1$$

for every (x, y) in the support of $\mathcal{D}$. Equivalently, $\dfrac{w^\star}{\|w^\star\|_1}$ has ℓ_1-normalized margin at least $\dfrac{1}{\|w^\star\|_1}$. When a finite sample is used, write $S = ((x_1, y_1), \ldots, (x_n, y_n))$.

You may use without proof that sparse linear classifiers have VC dimension $O\Big(s \log\Big(\frac{ed}{s}\Big)\Big)$, so exact sparse ERM has realizable sample complexity $O\Big(s \log\Big(\frac{ed}{s}\Big) + \log\Big(\frac{1}{\delta}\Big)\Big)\Big/\varepsilon$ up to constant factors, but is computationally difficult when s is part of the input.

1. **(12 points) From sparse margin to a weak coordinate.**

   Assume additionally that $\|w^\star\|_\infty \le B$. For any distribution Q supported on examples satisfying the margin condition, prove that some $b \in \mathcal{B}$ has

   $$L_Q(b) \le \frac{1}{2} - \frac{1}{2\|w^\star\|_1},$$

   and hence

   $$L_Q(b) \le \frac{1}{2} - \frac{1}{2sB}.$$

   Then describe an $O(nd)$ weighted ERM weak learner for $\mathcal{B}$ on weighted examples

   $$(x_i, y_i, D_i)_{i=1}^n.$$

   Hint: relate weighted error to the signed correlation

   $$\sum_i D_i y_i b(x_i).$$

> **Goal.**
> We want to show that the sparse margin condition forces at least one signed coordinate predictor to have nontrivial edge. The key point is that a sparse linear predictor is a weighted combination of signed coordinate predictors.

Step 1: Take expectation of the margin condition.
Since Q is supported only on examples satisfying the margin condition, we have

$$y\langle w^\star, \phi(x)\rangle \geq 1$$

Q-almost surely. Taking expectation under Q gives

$$\mathbb{E}_Q[y\langle w^\star, \phi(x)\rangle] \geq 1.$$

Expanding the inner product,

$$\mathbb{E}_Q\left[y\sum_{j=1}^{d} w_j^\star \phi_j(x)\right] \geq 1.$$

By linearity of expectation,

$$\sum_{j=1}^{d} w_j^\star \mathbb{E}_Q[y\phi_j(x)] \geq 1.$$

Step 2: Rewrite using signed coordinates.
For each nonzero coordinate, write

$$w_j^\star = |w_j^\star|\text{sign}(w_j^\star).$$

Then

$$\sum_{j=1}^{d} |w_j^\star|\mathbb{E}_Q[y\text{sign}(w_j^\star)\phi_j(x)] \geq 1.$$

Divide by $\|w^\star\|_1$:

$$\sum_{j=1}^{d} \frac{|w_j^\star|}{\|w^\star\|_1}\mathbb{E}_Q[y\text{sign}(w_j^\star)\phi_j(x)] \geq \frac{1}{\|w^\star\|_1}.$$

The coefficients

$$\frac{|w_j^\star|}{\|w^\star\|_1}$$

are nonnegative and sum to 1. Therefore the left side is a weighted average of the signed coordinate correlations

$$\mathbb{E}_Q[y\text{sign}(w_j^\star)\phi_j(x)].$$

If every signed coordinate correlation were smaller than

$$\frac{1}{\|w^\star\|_1},$$

then the weighted average would also be smaller than this value. That would contradict the inequality above. Hence there exists some coordinate j such that

$$\mathbb{E}_Q[y\text{sign}(w_j^\star)\phi_j(x)] \geq \frac{1}{\|w^\star\|_1}.$$

Define

$$b(x) = \text{sign}(w_j^\star)\phi_j(x).$$

Then

$$\mathbb{E}_Q[yb(x)] \geq \frac{1}{\|w^\star\|_1}.$$

Step 3: Convert correlation into error.
Since $b(x), y \in \{-1, +1\}$,

$$\mathbf{1}[b(x) \neq y] = \frac{1 - yb(x)}{2}.$$

Therefore

$$L_Q(b) = \mathbb{E}_Q\left[\frac{1 - yb(x)}{2}\right] = \frac{1}{2} - \frac{1}{2}\mathbb{E}_Q[yb(x)].$$

Using the lower bound on correlation,

$$L_Q(b) \leq \frac{1}{2} - \frac{1}{2\|w^\star\|_1}.$$

This proves the first desired claim.

Step 4: Use sparsity and the coefficient bound.
Since $\|w^\star\|_0 \leq s$ and $\|w^\star\|_\infty \leq B$,

$$\|w^\star\|_1 = \sum_{j=1}^{d} |w_j^\star| \leq sB.$$

Hence

$$\frac{1}{2\|w^\star\|_1} \geq \frac{1}{2sB}.$$

Therefore

$$L_Q(b) \leq \frac{1}{2} - \frac{1}{2sB}.$$

The weak-learning advantage is

$$\boxed{\gamma = \frac{1}{2sB}.}$$

Weighted ERM weak learner for $\mathcal{B}$.
Given weighted examples

$$(x_i, y_i, D_i)_{i=1}^n,$$

the weighted error of b is

$$L_D(b) = \sum_{i=1}^{n} D_i \mathbf{1}[b(x_i) \neq y_i].$$

Since

$$\mathbf{1}[b(x_i) \neq y_i] = \frac{1 - y_i b(x_i)}{2},$$

we get

$$L_D(b) = \frac{1}{2} - \frac{1}{2}\sum_{i=1}^{n} D_i y_i b(x_i).$$

Thus minimizing weighted error is equivalent to maximizing the signed correlation

$$\sum_{i=1}^{n} D_i y_i b(x_i).$$

For each coordinate j, compute

$$c_j = \sum_{i=1}^{n} D_i y_i \phi_j(x_i).$$

For the signed coordinate $b_{j,\sigma}(x) = \sigma\phi_j(x)$, the correlation is

$$\sum_{i=1}^{n} D_i y_i \sigma \phi_j(x_i) = \sigma c_j.$$

The best sign is

$$\sigma_j^\star = \text{sign}(c_j),$$

and the best coordinate is

$$j^\star = \arg\max_j |c_j|.$$

The weak learner returns

$$\boxed{b(x) = \text{sign}(c_{j^\star})\phi_{j^\star}(x).}$$

Computing all c_j requires scanning n examples for each of d coordinates, so the runtime is

$$\boxed{O(nd).}$$

2. **(14 points) Boosting guarantee and comparison with sparse ERM.**

Run AdaBoost over $\mathcal{B}$. At every round, the reweighted distribution is supported on the same margin-realizable sample, so the previous part applies with

$$\gamma = \frac{1}{2sB}.$$

Prove the exponential training-error bound, choose T so that the training error is zero, and then use the sparse-linear VC bound, applied with the number of activated coordinates in place of s, to obtain a realizable generalization guarantee.

State the resulting sample complexity up to logarithmic factors, and compare it with exact sparse ERM. Your comparison should identify the statistical price paid by boosting, the computational advantage, and the role of B.

AdaBoost weak edge.
From Part A.1, at every boosting round the current weighted distribution is supported on the same examples that satisfy the margin condition. Therefore the coordinate weak learner can always find a coordinate predictor with error at most

$$\frac{1}{2} - \gamma,$$

where

$$\gamma = \frac{1}{2sB}.$$

Exponential training-error bound.
The standard AdaBoost training-error bound says that if every weak learner has edge at least γ, then the final boosted classifier H_T satisfies

$$\widehat{L}_S(H_T) \leq \exp(-2\gamma^2 T).$$

Substituting

$$\gamma = \frac{1}{2sB},$$

we get

$$2\gamma^2 = 2\left(\frac{1}{2sB}\right)^2 = \frac{1}{2s^2B^2}.$$

Hence

$$\boxed{\widehat{L}_S(H_T) \leq \exp\left(-\frac{T}{2s^2B^2}\right).}$$

Choosing T for zero training error.
To guarantee zero training error on a sample of size n, it is enough to make

$$\widehat{L}_S(H_T) < \frac{1}{n}.$$

It suffices that

$$\exp\left(-\frac{T}{2s^2B^2}\right) < \frac{1}{n}.$$

Taking logarithms gives

$$-\frac{T}{2s^2B^2} < -\log n.$$

Therefore it is enough to choose

$$T > 2s^2B^2 \log n.$$

Thus

$$\boxed{T = O(s^2B^2 \log n)}$$

boosting rounds are enough to reach zero training error.

Generalization using activated coordinates.
The boosted classifier has the form

$$H_T(x) = \operatorname{sign}\left(\sum_{t=1}^{T} \alpha_t b_t(x)\right).$$

Each b_t is a signed coordinate predictor. Therefore the final classifier is a linear classifier using at most T activated coordinates. It may use fewer than T coordinates if some coordinates repeat, but T is always a valid upper bound.

The sparse-linear VC bound with sparsity T gives

$$\text{VCdim} = O\left(T \log \frac{ed}{T}\right).$$

Since the boosted classifier reaches zero empirical error, a realizable generalization bound gives sample complexity

$$n = \widetilde{O}\left(\frac{T \log(ed/T) + \log(1/\delta)}{\varepsilon}\right).$$

Substituting

$$T = O(s^2 B^2 \log n),$$

and hiding logarithmic factors, the sample complexity becomes

$$\boxed{n = \widetilde{O}\left(\frac{s^2 B^2 \log d + \log(1/\delta)}{\varepsilon}\right).}$$

Comparison with exact sparse ERM.
Exact sparse ERM with sparsity s has realizable sample complexity

$$\widetilde{O}\left(\frac{s \log(ed/s) + \log(1/\delta)}{\varepsilon}\right).$$

Boosting gives approximately

$$\widetilde{O}\left(\frac{s^2 B^2 \log d + \log(1/\delta)}{\varepsilon}\right).$$

Therefore boosting pays a statistical price. Instead of depending roughly linearly on s, the bound depends roughly on $s^2 B^2$. This is because boosting may activate as many as

$$T = O(s^2 B^2 \log n)$$

coordinates before reaching zero training error.

Computational advantage.
Exact sparse ERM requires searching over supports, which is computationally difficult when s is part of the input. In contrast, each coordinate weak-learning step takes only

$$O(nd)$$

time. Therefore the total boosting runtime is approximately

$$O(Tnd) = \widetilde{O}(s^2 B^2 nd).$$

This is polynomial when s and B are not too large.

Role of B.
The coefficient bound B controls the ℓ_1 norm:

$$\|w^\star\|_1 \le sB.$$

The weak edge is really controlled by the ℓ_1-normalized margin:

$$\gamma = \frac{1}{2\|w^\star\|_1}.$$

Bounding $\|w^\star\|_\infty$ by B converts this into a polynomial lower bound

$$\gamma \geq \frac{1}{2sB}.$$

Without such a coefficient bound, the realizing sparse vector could have exponentially large coefficients, the ℓ_1 margin could be exponentially small, and AdaBoost could require exponentially many rounds.

3. **(9 points) Why the coefficient bound matters.**

For odd $s = 2m+1$, set $d = s$. Construct a distribution supported on s labeled examples with $y = +1$ and $\varphi(x) \in \{-1, +1\}^s$ such that some $w^\star \in \mathbb{R}^s$ has margin 1, but every coordinate predictor has edge at most $2^{-\Omega(s)}$. Also show that the realizing vector in your construction satisfies

$$\|w^\star\|_\infty = 2^{\Omega(s)}.$$

Hint: index rows by $0, (1,+), (1,-), \ldots, (m,+), (m,-)$ with weights

$$q_0 = 1, \qquad q_{r,+} = q_{r,-} = 2^{r-1}.$$

It suffices to build an invertible $s \times s$ sign matrix whose every column has q-weighted sum 1; try a base column with $(+,-)$ on every pair, columns that flip one pair, and carry columns with $(-,-)$ on earlier pairs, $(+,+)$ on one pair, and $(+,-)$ later. Prove realizability, compute the best coordinate edge, and explain why this rules out any polynomial-in-s AdaBoost guarantee based only on sparsity.

Construction.
Let

$$s = 2m+1, \qquad d = s.$$

We construct s examples, all with label

$$y = +1.$$

Index the rows by

$$0,\ (1,+), (1,-), \ldots, (m,+), (m,-).$$

Define row weights

$$q_0 = 1, \qquad q_{r,+} = q_{r,-} = 2^{r-1}.$$

The total weight is

$$Q = 1 + \sum_{r=1}^{m} 2 \cdot 2^{r-1} = 1 + \sum_{r=1}^{m} 2^r = 2^{m+1} - 1.$$

The distribution assigns probability

$$\frac{q_i}{Q}$$

to row i.
We now construct an $s \times s$ sign matrix $A \in \{-1, +1\}^{s \times s}$. Each row of A is a feature vector $\phi(x)$.

Base column.
Define the base column c^0 by

$$c_0^0 = +1,$$

and for every pair r,

$$c_{r,+}^0 = +1, \qquad c_{r,-}^0 = -1.$$

Its q-weighted sum is

$$1 + \sum_{r=1}^{m} 2^{r-1}(+1) + 2^{r-1}(-1) = 1.$$

Flip columns.
For each $r \in [m]$, define a flip column f^r by setting

$$f_0^r = +1.$$

On pair r, flip the base signs:

$$f_{r,+}^r = -1, \qquad f_{r,-}^r = +1.$$

On every other pair $t \neq r$, use the base pattern:

$$f_{t,+}^r = +1, \qquad f_{t,-}^r = -1.$$

Every pair still cancels in the weighted sum, and the row 0 contributes 1. Therefore

$$\sum_i q_i f_i^r = 1.$$

Carry columns.
For each $r \in [m]$, define a carry column g^r by

$$g_0^r = -1.$$

For earlier pairs $t < r$,

$$g_{t,+}^r = g_{t,-}^r = -1.$$

For pair r,

$$g_{r,+}^r = g_{r,-}^r = +1.$$

For later pairs $t > r$,

$$g_{t,+}^r = +1, \qquad g_{t,-}^r = -1.$$

The later pairs cancel. The weighted sum is

$$-1 - \sum_{t<r} 2^t + 2^r.$$

Since

$$\sum_{t=1}^{r-1} 2^t = 2^r - 2,$$

the weighted sum equals

$$-1 - (2^r - 2) + 2^r = 1.$$

Therefore every column of A has q-weighted sum exactly 1.

Invertibility.
The assignment hint asks for an invertible sign matrix. The base, flip, and carry columns above are linearly independent. To see the main idea, suppose a linear combination of these columns equals zero:

$$ac^0 + \sum_{r=1}^{m} b_r f^r + \sum_{r=1}^{m} c_r g^r = 0.$$

First examine the sum of the two entries in each pair $(r,+),(r,-)$. The base and flip columns have pair-sum 0. The carry columns have pair-sum -2 on earlier pairs, $+2$ on their own pair, and 0 on later pairs. This gives a triangular system in the carry coefficients c_r. Starting from the last pair and moving backward gives

$$c_m = c_{m-1} = \cdots = c_1 = 0.$$

With the carry coefficients gone, only the base and flip columns remain. Looking at pairwise differences and the row 0 then forces all flip coefficients b_r and the base coefficient a to be zero. Thus the columns are linearly independent, so A is invertible.

Realizability with margin 1.
Since A is invertible, there exists a vector $w^\star \in \mathbb{R}^s$ satisfying

$$Aw^\star = \mathbf{1}.$$

Therefore for every row/example i,

$$\langle w^\star, \phi(x_i)\rangle = 1.$$

Since every label is $y_i = +1$,

$$y_i\langle w^\star, \phi(x_i)\rangle = 1.$$

Thus the distribution is realizable with margin exactly 1.

Every coordinate has exponentially small edge.
Since all labels are $+1$, the correlation of coordinate j is

$$\mathbb{E}[y\phi_j(x)] = \mathbb{E}[\phi_j(x)].$$

Every column of A has q-weighted sum 1. Therefore, under the normalized distribution,

$$\mathbb{E}[\phi_j(x)] = \frac{1}{Q}.$$

The edge of the positive coordinate predictor is

$$\frac{1}{2}\mathbb{E}[y\phi_j(x)] = \frac{1}{2Q}.$$

The negative signed coordinate has correlation $-1/Q$, so it is worse. Thus the best signed coordinate edge is

$$\frac{1}{2Q}.$$

Since

$$Q = 2^{m+1} - 1,$$

we have

$$\frac{1}{2Q} = 2^{-\Theta(m)} = 2^{-\Theta(s)}.$$

Hence every coordinate predictor has edge at most

$$\boxed{2^{-\Omega(s)}.}$$

The realizing vector has exponentially large coefficients.
Because every column has q-weighted sum 1,

$$q^\top A = (1, 1, \ldots, 1).$$

Since

$$Aw^\star = \mathbf{1},$$

multiplying by $q^\top$ gives

$$q^\top Aw^\star = q^\top \mathbf{1}.$$

The left side is

$$\sum_{j=1}^{s} w_j^\star,$$

and the right side is

$$Q.$$

Therefore

$$\sum_{j=1}^{s} w_j^\star = Q.$$

Hence

$$s\|w^\star\|_\infty \geq \left|\sum_{j=1}^{s} w_j^\star\right| = Q.$$

So

$$\|w^\star\|_\infty \geq \frac{Q}{s} = \frac{2^{m+1}-1}{2m+1} = 2^{\Omega(s)}.$$

Therefore

$$\boxed{\|w^\star\|_\infty = 2^{\Omega(s)}.}$$

Why this rules out a sparsity-only AdaBoost guarantee.
This example is realizable by an s-sparse vector with margin 1, but every coordinate predictor has only exponentially small edge. AdaBoost would need roughly

$$T = \Omega\left(\frac{1}{\gamma^2}\right)$$

rounds to get a strong training-error guarantee. If

$$\gamma = 2^{-\Omega(s)},$$

then

$$\frac{1}{\gamma^2} = 2^{\Omega(s)}.$$

Thus sparsity alone cannot guarantee polynomially many boosting rounds. The coefficient bound B is necessary because it controls the ℓ_1 margin and prevents this exponentially small edge phenomenon.

4. **(10 points) Experiment: sparse boosting versus a convex surrogate.**

Implement AdaBoost over the coordinate class $\mathcal{B}$. Compare an easy sparse-margin distribution of your choice with the construction from the previous part. Report training error, exponential loss, observed edge, support size, and normalized margin over rounds. Compare AdaBoost with one convex surrogate method, for example logistic regression or hinge-loss minimization with an ℓ_1 constraint or penalty. Explain what the experiment illustrates about support sparsity, coefficient size, ℓ_1 margin, and computational tractability.

Experiment artifact.
The experiment for this part is implemented in the supporting Python file

`code/boosting_experiment.py`.

I do not include the full code in the main write-up because the assignment asks for a single PDF write-up plus supporting repository artifacts. The Python file should be placed in the assignment repository, and this write-up should refer to it as the reproducibility artifact.

Methods compared.
The first method is AdaBoost over the signed coordinate class

$$\mathcal{B} = \{b_{j,\sigma}(x) = \sigma\phi_j(x) : j \in [d],\ \sigma \in \{-1, +1\}\}.$$

At each round, the weak learner computes the weighted signed correlation

$$c_j = \sum_i D_i y_i \phi_j(x_i)$$

for every coordinate and chooses the coordinate/sign pair with largest $|c_j|$.
The second method is a convex surrogate method: logistic regression with an ℓ_1 penalty. Its objective is of the form

$$\min_w \frac{1}{n}\sum_{i=1}^{n} \log\left(1 + \exp(-y_i\langle w, x_i\rangle)\right) + \lambda\|w\|_1.$$

This is not the same as AdaBoost and not the same as sparse 0-1 ERM. It is included because it is a tractable convex surrogate that encourages sparse coefficients.

Datasets.
The experiment compares two distributions.

(a) **Easy sparse-margin data.** In this case, the labels are generated by a sparse vector with small coefficients. This means $\|w^\star\|_\infty$ is small, so $\|w^\star\|_1$ is also controlled. Therefore the ℓ_1-normalized margin is not too small, and AdaBoost should quickly find useful coordinate predictors.

(b) **Hard construction from Part A.3.** In this case, the data is realizable with margin 1, but the realizing vector has exponentially large coefficients. As proved in Part A.3, every coordinate edge is only

$$2^{-\Omega(s)}.$$

Therefore AdaBoost should make much slower progress.

Metrics reported.
Over boosting rounds, the Python script reports:

- **training error**
$$\frac{1}{n}\sum_{i=1}^{n}\mathbf{1}[H_T(x_i)\neq y_i],$$
- **exponential loss**
$$\frac{1}{n}\sum_{i=1}^{n}\exp(-y_iF_T(x_i)),$$
where $F_T(x)=\sum_{t=1}^{T}\alpha_t b_t(x)$,
- **observed edge**
$$\frac{1}{2}-\text{weighted error},$$
- **support size**, meaning the number of distinct coordinates selected so far,
- **normalized margin**
$$\frac{\min_i y_iF_T(x_i)}{\sum_{t=1}^{T}|\alpha_t|}.$$

Expected behavior on the easy sparse-margin data.
On the easy sparse-margin distribution, AdaBoost should reach zero training error in a small number of rounds. The observed edges should be noticeably bounded away from 0. The support size should grow slowly because only a small number of coordinates are genuinely useful. Logistic regression with an ℓ_1 penalty should also perform well because the true separator has small support and moderate coefficient size.

Expected behavior on the hard construction.

On the hard construction, the true separator exists, and it even has margin 1. However, the useful separator has very large coefficients:

$$\|w^\star\|_\infty = 2^{\Omega(s)}.$$

This makes the ℓ_1-normalized margin tiny. As a result, the best coordinate edge is only exponentially small:

$$\gamma = 2^{-\Omega(s)}.$$

Therefore AdaBoost should improve slowly. The experiment should show small observed edges and slow decay of the exponential loss.

Interpretation.
The experiment illustrates that support sparsity is not the whole story. In the easy case, sparsity and bounded coefficients work together: the separator uses few coordinates and has a reasonable ℓ_1 margin. In that setting, coordinate boosting is computationally efficient and statistically meaningful.
In the hard construction, the support size is still only s, and the data is margin-realizable. But the coefficient size is exponentially large. This destroys the coordinate weak-learning guarantee because the ℓ_1-normalized margin becomes exponentially small. AdaBoost is not failing because no separator exists; it is struggling because no individual coordinate gives a useful weak advantage.
The convex surrogate method is computationally tractable because it solves a convex optimization problem instead of searching over supports. However, logistic regression with an ℓ_1 penalty is not the same as exact sparse ERM and is not guaranteed to return a proper s-sparse classifier. It encourages sparse-looking coefficients but does not directly enforce a hard support constraint.
Therefore the experiment supports the main lesson of Part A:

sparsity helps statistically, but coefficient size and ℓ_1 margin control boosting.

It also shows the computational tradeoff:

coordinate boosting is efficient when weak edges exist, while convex surrogates are efficient but different.

2. Part B: Agnostic Halfspace Hardness via Boosting

Part A used boosting constructively; here boosting is a reduction tool.

Let $\mathcal{X}_d = \mathbb{R}^d$ and $\mathcal{Y} = \{-1, +1\}$. Let $\mathcal{H}_d$ be the class of affine halfspaces

$$h_{w,b}(x) = \text{sign}(\langle w, x\rangle + b),$$

with $\text{sign}(z) = +1$ for $z \geq 0$. Let $\mathcal{I}_{d,k}$ be the class of intersections of k halfspaces: the output is $+1$ iff all k halfspaces output $+1$. For a size function $k = k(d)$, polynomial-size means $k(d) \leq d^c$ for some fixed constant c, and $k9d) = w(1)$ means $k(d) \to \infty$

Use these black-box hardness facts: under standard uSVP hardness, intersections of d^r affine halfspaces are not efficiently PAC learnable in the realizable case, even improperly, for every fixed $r > 0$; under RSAT, the same is true for intersections of $k(d) = \omega(1)$ affine halfspaces.

1. **(10 points) A weak halfspace inside an intersection.**

   Prove: if $\mathcal{D}$ is realizable by some $g \in \mathcal{I}_{d,k}$, then some affine halfspace has error at most

   $$\frac{1}{2} - \frac{1}{2k^2}.$$

   Hint: split on $p = \mathbb{P}[y = +1]$. For small p, use the constant -1 halfspace; for large p, average over the k halfspaces defining the realizing intersection.

Setup.
Suppose $\mathcal{D}$ is realizable by an intersection

$$g(x) = h_1(x) \wedge h_2(x) \wedge \cdots \wedge h_k(x),$$

where each h_j is an affine halfspace. The intersection predicts $+1$ exactly when all k halfspaces predict $+1$.
Let

$$p = \mathbb{P}[y = +1].$$

We split into two cases.

Case 1: $p \leq \frac{1}{2} - \frac{1}{2k^2}$**.**
Use the constant -1 halfspace. This is an affine halfspace because we can take

$$w = 0, \qquad b = -1.$$

Then

$$h(x) = \text{sign}(-1) = -1$$

for every x.
This classifier makes mistakes exactly on the positive examples. Therefore its error is

$$L_{\mathcal{D}}(h) = p.$$

By the case assumption,

$$p \leq \frac{1}{2} - \frac{1}{2k^2}.$$

Hence the desired halfspace exists in this case.

Case 2: $p > \frac{1}{2} - \frac{1}{2k^2}$**.**
Now consider the k halfspaces $h_1, \ldots, h_k$ defining the realizing intersection.
On a positive example, the intersection outputs $+1$. Therefore all k halfspaces must output $+1$. Since the true label is $+1$, all k halfspaces are correct on every positive example.
On a negative example, the intersection outputs -1. This means at least one of the k halfspaces outputs -1. Since the true label is -1, at least one of the k halfspaces is correct on every negative example.
Therefore the average accuracy over the k halfspaces is at least

$$p \cdot 1 + (1 - p) \cdot \frac{1}{k}.$$

Hence at least one halfspace has accuracy at least

$$p + \frac{1-p}{k}.$$

Its error is at most

$$1 - \left(p + \frac{1-p}{k}\right) = (1-p)\left(1 - \frac{1}{k}\right).$$

Since

$$p > \frac{1}{2} - \frac{1}{2k^2},$$

we have

$$1 - p < \frac{1}{2} + \frac{1}{2k^2}.$$

Therefore

$$L_{\mathcal{D}}(h) \leq \left(\frac{1}{2} + \frac{1}{2k^2}\right)\left(1 - \frac{1}{k}\right).$$

Expanding,

$$\left(\frac{1}{2} + \frac{1}{2k^2}\right)\left(1 - \frac{1}{k}\right) = \frac{1}{2} - \frac{1}{2k} + \frac{1}{2k^2} - \frac{1}{2k^3}.$$

We compare this to

$$\frac{1}{2} - \frac{1}{2k^2}.$$

It is enough to show

$$-\frac{1}{2k} + \frac{1}{2k^2} - \frac{1}{2k^3} \leq -\frac{1}{2k^2}.$$

Multiplying by $2k^3 > 0$ gives

$$-k^2 + k - 1 \leq -k.$$

Equivalently,

$$-k^2 + 2k - 1 \leq 0.$$

This is

$$-(k-1)^2 \leq 0,$$

which is true.
Hence in this case also, some affine halfspace has error at most

$$\frac{1}{2} - \frac{1}{2k^2}.$$

Conclusion.
In both cases, there exists an affine halfspace h such that

$$\boxed{L_{\mathcal{D}}(h) \leq \frac{1}{2} - \frac{1}{2k^2}.}$$

Therefore every realizable intersection of k halfspaces contains a weak affine halfspace with advantage

$$\boxed{\gamma = \frac{1}{2k^2}.}$$

2. **(10 points) From an agnostic learner to a weak learner.**

Suppose, hypothetically, that affine halfspaces are efficiently properly agnostically PAC learnable. Use the previous lemma to construct a weak learner for $\mathcal{I}_{d,k}$ in the realizable case. Give γ such that the returned halfspace has error at most

$$\frac{1}{2} - \gamma,$$

and explain why the weak learner is polynomial-time when $k(d) \leq d^c$ for a fixed constant c.

Hypothetical assumption.
Assume affine halfspaces are efficiently properly agnostically PAC learnable. This means that for any distribution and any accuracy parameter $\alpha > 0$, the learner returns an affine halfspace whose error is within α of the best affine halfspace error:

$$L(h) \leq \inf_{h' \in \mathcal{H}_d} L(h') + \alpha.$$

Use Part B.1.
If the distribution is realizable by an intersection of k halfspaces, then Part B.1 shows that

$$\inf_{h' \in \mathcal{H}_d} L(h') \leq \frac{1}{2} - \frac{1}{2k^2}.$$

Run the hypothetical agnostic halfspace learner with

$$\alpha = \frac{1}{4k^2}.$$

Then it returns a proper affine halfspace h satisfying

$$L(h) \leq \frac{1}{2} - \frac{1}{2k^2} + \frac{1}{4k^2}.$$

Therefore

$$L(h) \leq \frac{1}{2} - \frac{1}{4k^2}.$$

Thus this gives a weak learner with advantage

$$\boxed{\gamma = \frac{1}{4k^2}.}$$

Polynomial-time condition.
If

$$k(d) \leq d^c$$

for a fixed constant c, then

$$\frac{1}{\gamma} = 4k^2 \leq 4d^{2c}.$$

Therefore the accuracy requirement

$$\alpha = \frac{1}{4k^2}$$

is inverse-polynomial in d. Since the hypothetical agnostic learner is efficient, running it with inverse-polynomial accuracy takes polynomial time.
Hence for polynomially bounded $k(d)$, the assumed agnostic halfspace learner gives a polynomial-time weak learner for intersections of k halfspaces in the realizable case.

3. **(12 points) Boosting the weak learner.**

Use AdaBoost and the boosted-halfspace VC bound $\widetilde{O}(Td)$ to obtain a realizable learner for $\mathcal{I}_{d,k}$. At every round, the weighted distribution is supported on examples still realized by the same intersection. State the sample and runtime dependence up to logarithmic factors.

Weak learner advantage.
From Part B.2, the hypothetical agnostic learner gives a weak learner with advantage

$$\gamma = \frac{1}{4k^2}.$$

That is, at every boosting round the weak learner returns an affine halfspace with weighted error at most

$$\frac{1}{2} - \gamma.$$

This remains true at every AdaBoost round because reweighting the sample does not change the labels or the fact that the examples are realized by the same intersection of k halfspaces.

AdaBoost training error.
AdaBoost gives

$$\widehat{L}_S(H_T) \leq \exp(-2\gamma^2 T).$$

Since

$$\gamma = \frac{1}{4k^2},$$

we have

$$\gamma^2 = \frac{1}{16k^4}$$

and

$$2\gamma^2 = \frac{1}{8k^4}.$$

Therefore

$$\widehat{L}_S(H_T) \leq \exp\left(-\frac{T}{8k^4}\right).$$

To make the training error less than $1/n$, it is enough to choose

$$\exp\left(-\frac{T}{8k^4}\right) < \frac{1}{n}.$$

Taking logs gives

$$T > 8k^4 \log n.$$

So

$$\boxed{T = O(k^4 \log n)}$$

rounds are enough to get zero training error.

Generalization bound.
The boosted classifier is a weighted vote of T affine halfspaces:

$$H_T(x) = \operatorname{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right).$$

The assignment gives the boosted-halfspace VC bound

$$\widetilde{O}(Td).$$

Since the boosted classifier achieves zero training error, the realizable sample complexity is

$$n = \widetilde{O}\left(\frac{Td + \log(1/\delta)}{\varepsilon}\right).$$

Substituting

$$T = \widetilde{O}(k^4),$$

gives

$$\boxed{n = \widetilde{O}\left(\frac{k^4 d + \log(1/\delta)}{\varepsilon}\right).}$$

Runtime dependence.
The runtime is the number of boosting rounds times the runtime of the agnostic halfspace learner used as a weak learner. Thus,

$$\text{Runtime} = \widetilde{O}(k^4) \cdot \text{Runtime}_{\text{agnostic halfspace learner}}.$$

If

$$k(d) \le d^c$$

for fixed c, then k^4 is polynomial in d. Therefore, under the hypothetical assumption that proper agnostic halfspace learning is efficient, the boosted learner is also polynomial time.

4. **(8 points) Consequence for agnostic halfspaces.**

Prove the implication: if affine halfspaces are efficiently properly agnostically PAC learnable, then intersections of $k(d) \le d^c$ affine halfspaces are efficiently learnable in the realizable case, for every fixed c.

Explain the contradiction with the black-box hardness facts by taking $k(d) = d^r$ under uSVP, or any polynomially bounded $k(d) = \omega(1)$ under RSAT. Conclude the implication for efficient proper agnostic PAC learning of halfspaces.

Implication.
Suppose affine halfspaces are efficiently properly agnostically PAC learnable. Then, by Part B.2, we can use the agnostic learner to build a weak learner for intersections of k halfspaces with advantage

$$\gamma = \frac{1}{4k^2}.$$

By Part B.3, AdaBoost converts this weak learner into a strong realizable learner for intersections of k halfspaces using

$$T = \widetilde{O}(k^4)$$

rounds and sample complexity

$$\widetilde{O}\left(\frac{k^4 d + \log(1/\delta)}{\varepsilon}\right).$$

If

$$k(d) \leq d^c$$

for a fixed constant c, then

$$k^4 d \leq d^{4c+1}.$$

Therefore the sample complexity is polynomial in d, $1/\varepsilon$, and $\log(1/\delta)$. The runtime is also polynomial because it is polynomially many calls to the assumed efficient agnostic halfspace learner.
Hence

efficient proper agnostic learning of halfspaces

implies

efficient realizable learning of intersections of polynomially many halfspaces.

Contradiction under uSVP.
The black-box hardness fact says that under standard uSVP hardness, intersections of

$$d^r$$

affine halfspaces are not efficiently PAC learnable in the realizable case, even improperly, for every fixed $r > 0$.
But $k(d) = d^r$ is polynomially bounded. Therefore the implication above would give an efficient realizable learner for a class that the hardness assumption says is not efficiently learnable. This is a contradiction.

Contradiction under RSAT.
The black-box hardness fact under RSAT says that intersections of

$$k(d) = \omega(1)$$

affine halfspaces are not efficiently PAC learnable in the realizable case. If such a $k(d)$ is polynomially bounded, then the implication above again gives an efficient realizable learner, contradicting the hardness result.

Final conclusion.
Therefore, under these standard hardness assumptions, affine halfspaces cannot be efficiently properly agnostically PAC learnable. In short:

Efficient proper agnostic PAC learning of affine halfspaces would imply

efficient learning of hard intersection classes, which contradicts the hardness facts.

3. Part C: AI Usage Report

Write a short report describing how you used AI in this assignment. Do not just list tools; explain what role AI played in your work and how you checked the result. Address:

1. Describe the parts of the assignment for which you used AI.
2. Describe concrete AI suggestions you accepted and explain why.
3. Describe concrete AI suggestions you rejected or substantially modified.
4. Describe how you verified the correctness of what you submitted.

Also describe concrete updates to your AI workflow that resulted from this assignment. Explain the 5 most recent changes you made to your AI workflow and why.

AI usage report.
I used AI assistance for this assignment as a collaborator for proof planning, algebra checking, experiment design, and exposition. I did not treat AI output as a final authority. I checked each proof step manually and rewrote the arguments into my own structure before including them in this write-up.

Parts of the assignment where I used AI.
For Part A, I used AI to help organize the relationship between the sparse margin condition and the existence of a weak coordinate predictor. The main useful idea was to rewrite the margin condition as a weighted average over signed coordinate predictors using the ℓ_1-normalized coefficients of $w^\star$.
For Part A.2, I used AI to check the AdaBoost training-error calculation and the dependence of the number of boosting rounds on s, B, and $\log n$.
For Part A.3, I used AI to help decode the construction suggested by the hint. In particular, AI helped organize the base column, flip columns, and carry columns. I then checked the weighted sums, invertibility argument, edge calculation, and coefficient-size lower bound myself.
For Part A.4, I used AI to draft the structure of a Python experiment comparing AdaBoost over signed coordinates with ℓ_1-regularized logistic regression. The actual code is saved separately as

`code/boosting_experiment.py`.

For Part B, I used AI to help structure the reduction from hypothetical agnostic halfspace learning to weak learning and then to boosted learning for intersections of halfspaces. I checked the case analysis and algebra carefully.

Concrete AI suggestions I accepted.
I accepted the suggestion to prove Part A.1 by viewing

$$\sum_j |w_j^\star| \mathbb{E}_Q[y \mathrm{sign}(w_j^\star)\phi_j(x)]$$

as a weighted average. This was useful because it directly shows that at least one signed coordinate must have correlation at least $1/\|w^\star\|_1$.
I accepted the suggestion to express the weighted coordinate weak learner through the correlations

$$c_j = \sum_i D_i y_i \phi_j(x_i).$$

This is correct because minimizing weighted error is equivalent to maximizing signed correlation. I accepted the suggestion to split Part B.1 into the cases

$$p \leq \frac{1}{2} - \frac{1}{2k^2}$$

and

$$p > \frac{1}{2} - \frac{1}{2k^2}.$$

This matched the hint and made the proof clean.

Concrete AI suggestions I rejected or modified.
I rejected earlier overly vague statements like "boosting gives a polynomial learner" because they did not specify the dependence on k, d, ε, and δ. I modified the write-up to include the boosting-round calculation

$$T = \widetilde{O}(k^4)$$

and the resulting sample complexity

$$\widetilde{O}\left(\frac{k^4 d + \log(1/\delta)}{\varepsilon}\right).$$

I also modified the Part A.3 construction explanation. A short version that only claimed invertibility was not enough. I added a more explicit explanation using pair sums and the triangular structure of the carry columns.
For the experiment section, I rejected including the full Python code inside the main write-up because that would make the PDF too long. Instead, I refer to the supporting Python file and explain what metrics the code reports.

How I verified correctness.
For the proof parts, I checked each inequality line by line. In Part A.1, I verified that the edge definition matches the identity

$$L_Q(b) = \frac{1}{2} - \frac{1}{2}\mathbb{E}_Q[yb(x)].$$

In Part A.2, I verified the substitution

$$\gamma = \frac{1}{2sB}$$

into the AdaBoost bound.
In Part A.3, I manually checked that every constructed column has q-weighted sum 1. I also checked that

$$Q = 2^{m+1} - 1$$

and therefore the best coordinate edge is

$$\frac{1}{2Q} = 2^{-\Theta(s)}.$$

I checked the coefficient lower bound by multiplying

$$Aw^\star = \mathbf{1}$$

on the left by $q^\top$.
In Part B, I checked the algebra showing that

$$\left(\frac{1}{2}+\frac{1}{2k^2}\right)\left(1-\frac{1}{k}\right)\le\frac{1}{2}-\frac{1}{2k^2}.$$

I also verified that allowing the agnostic learner accuracy slack changes the weak advantage from

$$\frac{1}{2k^2}$$

to

$$\frac{1}{4k^2}.$$

For the experiment, I checked that the Python file reports the metrics requested in the prompt: training error, exponential loss, observed edge, support size, and normalized margin.

AI workflow updates.
The five most recent updates I made to my AI workflow are:

1. I added a checklist item requiring every proof to state the exact assumption being used. This helped avoid using the coefficient bound B before it was introduced.
2. I added a checklist item requiring every asymptotic statement to identify the relevant variables. This helped make the boosting and sample-complexity comparisons clearer.
3. I added a step asking AI to test constructions against the problem hint before accepting them. This was useful in Part A.3 because the weighted column sums had to be exactly 1.
4. I added a requirement that code artifacts have a short description explaining how they relate to the write-up. This is why the experiment is referred to as `code/boosting_experiment.py`.
5. I added a final verification pass where I compare the final write-up against the assignment prompt point by point. This helped make sure the solution addressed the proofs, comparisons, experiment explanation, and AI usage report.

I did not use Lean for this assignment. Lean would be possible for some algebraic subclaims, but the main assignment depends on AdaBoost guarantees, VC bounds, and black-box hardness assumptions, so a Lean formalization would not be the best use of time. Python is the more useful supporting artifact for this assignment because Part A.4 explicitly asks for an experiment.